Wrong Names in Natural Language Autoencoder Explanations

A project report submitted for the course

CS787: Generative Artificial Intelligence

by

Tanvi Pooranmal Meena

Roll No. 231085

Department of Mechanical Engineering

Course instructor

Prof. Arnab Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology Kanpur

November 2026

Certificate

This is to certify that the project report titled *"Wrong Names in Natural Language Autoencoder Explanations"*, submitted by **Tanvi Pooranmal Meena** (Roll No. 231085) for the course CS787: Generative Artificial Intelligence, is a record of work carried out by the student under my guidance during the academic year 2026–27, Semester I.

Date: _____

Place: Kanpur

_____

Prof. Arnab Bhattacharya
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

Declaration

I declare that this report describes my own work, carried out for the course CS787: Generative Artificial Intelligence. Where I have used the ideas, models, data or code of others, I have cited the source. The report has not been submitted, in whole or in part, for any other course or degree.

I used an AI coding assistant to help write and run the code and analysis scripts, and to help draft this report. I chose the questions, directed the experiments, checked the results, and I take responsibility for everything reported here.

Date: _____

Place: Kanpur

_____

Tanvi Pooranmal Meena
Roll No. 231085
Department of Mechanical Engineering

Acknowledgements

I thank Prof. Arnab Bhattacharya for the course, and for two pieces of advice this report tries to follow: that a narrow question answered carefully is worth more than a broad one answered loosely, and that understanding why something behaves the way it does matters more than making it work.

This project would not exist without the authors of the Natural Language Autoencoders work, who released their trained models, their training code and unusually candid documentation. Everything here is built on their Qwen2.5-7B checkpoints.

The experiments ran on a laptop, on free Kaggle GPUs and on rented RunPod GPUs. The passages come from Wikipedia, and the name lists from GeoNames, WordNet and spaCy.

Abstract

A natural language autoencoder (NLA) explains an activation inside a language model by writing a paragraph about it, and is trained so that a second model can rebuild the activation from the paragraph. The explanations read well, but they sometimes name the wrong specific: a city, person or instrument that was never in the text. This report measures that failure on the released NLA for Qwen2.5-7B, using 6,000 Wikipedia passages about 60 entities, and asks how often it happens, why, and what removes it.

Four findings come out. First, specifics fade from the explanation faster than from the activation: sixteen tokens after an entity, the explanation names it 7.5% of the time while a linear probe still reads it 35% of the time, and 22% of explanations contain a wrong name of the same type. Second, the training objective barely notices: editing a wrong name out changes the reconstruction cosine by about 0.002. Third, agreement between sampled explanations separates true names from wrong ones; keeping only names that two of four samples share removes 74% of wrong names and keeps 77% of true ones, with no training. Fourth, fine-tuning the verbalizer to prefer names its other samples support cuts wrong names by a further 8% on held-out passages, while fine-tuning on reconstruction alone changes nothing.

The practical advice is simple: do not trust a name in a single NLA explanation, sample several and keep what they agree on.

Contents

Certificate i

Declaration ii

Acknowledgements iii

Abstract iv

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Question . . . 1
- 1.3 Contributions . . . 2

2 Background 3
- 2.1 Autoencoders . . . 3
- 2.2 NLA Design . . . 3
- 2.3 Specifics . . . 4

3 Setup 6
- 3.1 Data . . . 6
- 3.2 Activations . . . 6
- 3.3 Explanations . . . 7
- 3.4 Name Detector . . . 7
- 3.5 Probe Baseline . . . 8
- 3.6 Statistics . . . 8

4 Fading 9
- 4.1 Fade Curve . . . 9
- 4.2 Wrong Names . . . 9
- 4.3 Name Position . . . 11

5 Incentives 13
- 5.1 Edit Test . . . 13
- 5.2 Results . . . 13
- 5.3 Interpretation . . . 13

6 Agreement 15
- 6.1 Consensus . . . 15
- 6.2 Filter . . . 15
- 6.3 Verifiers . . . 15

List of Figures

2.1 The three models in a natural language autoencoder 4

4.1 The specific fades faster from the explanation than from the activation . 10

5.1 What each kind of name is worth to the reconstruction score 14

6.1 Names that more samples agree on are more often true 16

List of Tables

3.1 The 60 entities. 7

4.1 Reading the entity from the activation 10
4.2 What the explanations name 11
4.3 The verbalizer against an equally selective probe 11
4.4 Where names sit in the explanation 12

5.1 Reconstruction cosine before and after editing names out 14

6.1 Precision of a name by how many samples give it 16
6.2 The agreement filter as a precision–recall trade-off 17
6.3 Three ways to tell true names from wrong ones 17

1 Introduction

1.1 Motivation

A natural language autoencoder (NLA) takes one activation vector from inside a language model and writes a short paragraph describing what that vector contains [3]. A second model reads the paragraph and tries to rebuild the vector. If the rebuilt vector is close to the original, the paragraph must have carried most of the information, so it can be read as an explanation of the activation. The appeal is obvious: instead of a list of feature numbers, you get English.

The trouble is that English is persuasive. Here is a passage from Wikipedia, and what the released NLA for Qwen2.5-7B wrote about the activation one token after the word *cello*:

Passage (the model has read up to "son")

> *. . . It has since performed at the Wigmore Hall in London, and in many countries worldwide. A recording of the* ***cello*** *son|atas of Antonio Vivaldi . . .*

Explanation written by the NLA

> *Biographical music description format with British classical composer context . . . Final token "son" is a truncated word mid-expression ("the* ***piano*** *son"), part of a noun phrase listing the completed work's content ("the* ***piano*** *sonatas"), strongly expecting "atas" . . .*

Most of this is right. The explanation knows the text is about classical music in a British setting, it knows the current word is the first half of "sonatas", and it knows which letters come next. The one thing it gets wrong is the specific: the instrument was a cello, and the explanation says piano, twice, with no hint of doubt. A reader who trusts the explanation walks away believing the model was thinking about a piano.

This report is about that failure. I call the cello a *specific*: a named entity that is really in the text. The gist (classical music, a recording, sonatas) survives the round trip through language. The specific often does not, and what replaces it is a confident wrong name of the same kind.

1.2 Question

I ask one narrow question and try to answer it carefully:

> When an NLA explanation names an entity, how often is the name wrong, why does that happen, and what removes the wrong names?

The middle part matters most. The aim is to understand why the verbalizer behaves this way, not to ship a better one. So *how often* is a measurement against a baseline, *why* is

tested with an intervention, and the fixes are there mainly as checks on the explanation: if wrong names are free under the training objective, more training on that objective should not remove them, and a penalty on them should.

1.3 Contributions

Working with the released Qwen2.5-7B NLA and 6,000 Wikipedia passages about 60 entities, I find four things.

1. **Specifics fade from the explanation faster than from the activation.** At the entity's own token the explanation names it essentially every time. Sixteen tokens later it names it 7.5% of the time, while a linear probe on the same activations still reads it 35% of the time. Over the same stretch, between a fifth and a half of all explanations contain a wrong name of the same type (Chapter 4).
2. **The training objective barely notices a wrong name.** Editing the wrong names out of an explanation changes the reconstruction score by about 0.002 per name. Editing the true name out costs 0.21 at the entity's token and 0.016 one token later (Chapter 5).
3. **Agreement between samples separates right from wrong.** A name that appears in only one of four sampled explanations is right 6% of the time; a name that appears in all four is right 64% of the time. Keeping only names that at least two samples agree on removes 74% of the wrong names and keeps 77% of the true ones, with no training and no access to labels (Chapter 6).
4. **Fine-tuning on that signal helps, a little.** Rewarding explanations whose names the other samples support cuts wrong names by about 8% on held-out passages without losing true names. Rewarding reconstruction alone, or a contrastive version of it, changes nothing.

Along the way I found that the usage example published with the models reads activations one layer too early, which silently lowers reconstruction quality; Appendix B documents the bug and the check that confirmed the fix. Every table in this report is generated from saved outputs by one command.

Availability. Everything needed to check or rerun the work is in three places:

- Code, explanations, result tables and this report: https://github.com/tanvincible/nla-wrong-names
- Fine-tuned adapters: https://huggingface.co/tanvincible/nla-qwen2.5-7b-L20-av-speci
- Activations, caches and training samples: https://huggingface.co/datasets/tanvincible/nla-specifics-artifacts

2 Background

2.1 Autoencoders

An autoencoder is a pair of functions trained together: an encoder that squeezes an input into a smaller code, and a decoder that rebuilds the input from the code [4]. Nothing tells the encoder what the code should mean. The only pressure is that the decoder has to succeed, so the code ends up holding whatever the decoder needs most.

That last sentence is the key to this whole report. An autoencoder keeps what helps reconstruction and is indifferent to everything else. If two different inputs can be rebuilt almost equally well from the same code, the encoder has no reason to tell them apart.

2.2 NLA Design

A natural language autoencoder [3] is an autoencoder whose input is an activation vector from a language model and whose code is a paragraph of English. Figure 2.1 shows the three models involved.

Target model. The model being explained. Here it is Qwen2.5-7B-Instruct [7], and the activation is the residual stream after block 20 of 28, a vector of 3,584 numbers at one token position.

Verbalizer. A copy of the target model, fine-tuned to describe an activation. The vector is rescaled and placed into a fixed prompt as if it were the embedding of a single token; the verbalizer then writes an explanation, one token at a time. This is the encoder.

Reconstructor. Another fine-tuned copy, cut off after the extraction layer, with a linear layer on top. It reads the explanation and outputs a vector. This is the decoder.

In symbols, let $h \in \mathbb{R}^d$ be the activation, $V_\theta(z \mid h)$ the verbalizer's distribution over explanations z, and $R_\phi(z) \in \mathbb{R}^d$ the reconstructor's output. Both vectors are scaled to unit length before they are compared, so the error depends only on direction:

$$\left\| \frac{h}{\|h\|} - \frac{R_\phi(z)}{\|R_\phi(z)\|} \right\|^2 = 2\Big(1 - s(h, z)\Big), \qquad s(h, z) = \cos\Big(h, R_\phi(z)\Big). \tag{2.1}$$

I call s the reconstruction score and report it throughout.

The authors train in three stages. The reconstructor first learns to predict activations from text. The verbalizer first learns to imitate explanations that a large language model, called through an API, wrote for each position. Then the two are trained together: the verbalizer by reinforcement learning (GRPO [8]), with the reconstruction score as its reward, and the reconstructor by ordinary regression on whatever the verbalizer writes.

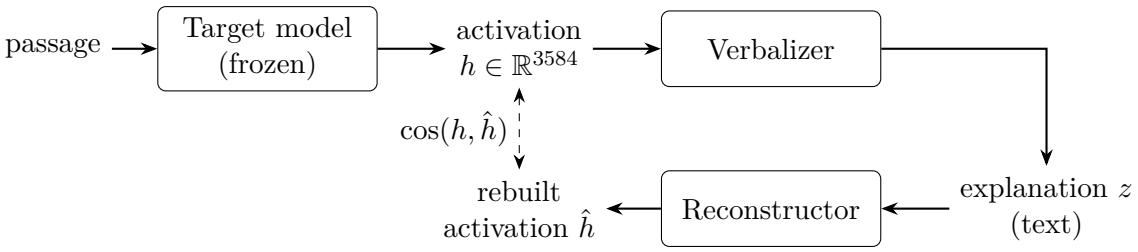


Figure 2.1: The three models in a natural language autoencoder. The target model reads a passage; one activation h is taken from its residual stream. The verbalizer turns h into text, and the reconstructor turns the text back into a vector. Training rewards a high cosine between h and $\hat{h}$.

In this last stage the verbalizer's objective is

$$\max_{\theta} \; \mathbb{E}_h \, \mathbb{E}_{z \sim V_\theta(\cdot|h)} \Big[\, s(h, z) \, \Big], \tag{2.2}$$

and nothing else. Whatever does not move s is invisible to training.

The explanations the released verbalizer writes have a steady three-part shape, visible in the example in Chapter 1. The first paragraph gives the genre and topic of the text. The second quotes or paraphrases the sentence being read. The third describes the final token and lists what is expected next. I use this shape later, to ask whether wrong names sit in the part that describes the context or the part that predicts the continuation.

2.3 Specifics

By a *specific* I mean a named entity that appears in the passage the target model read: a particular country, city, person, animal, food or instrument. An explanation can do three things with it.

- Name it. I call this a *true name*.
- Name a different entity of the same type that is not in the passage, as with the piano above. I call this a *wrong name*.
- Name nothing of that type, and describe the passage only in general terms.

To make this precise, let e be the entity in the passage, P the set of same-type names that occur in the passage, and $N(z)$ the set of same-type names in an explanation z. Then z names the true entity when $e \in N(z)$, and its wrong names are

$$W(z) = N(z) \setminus \Big(P \cup \{e\} \Big). \tag{2.3}$$

The third option is not a failure. If the activation no longer holds the entity, the faithful thing to say is nothing. The failure this report studies is the second one, because a wrong name is indistinguishable from a true one to anyone reading the explanation without the passage in front of them.

Specifics are the natural place to look for trouble. The gist of a passage (music, a biography, a list of recordings) is shared by thousands of training examples and moves the activation a long way, so the reconstructor depends on it and the verbalizer learns to report it. Which instrument it was moves the activation much less. Chapter 5 measures exactly how much less.

3 Setup

Everything in this report runs on the released NLA for Qwen2.5-7B-Instruct at layer 20. The target model, the verbalizer and the reconstructor are used exactly as published, except in the fine-tuning experiments, where a small adapter is added to the verbalizer. This chapter describes the data and the three measuring instruments: a detector for names, a probe that says what the activation still holds, and the statistics.

3.1 Data

I chose 60 entities, ten of each of six types (Table 3.1). Three types are proper nouns and three are common nouns, so that the results do not depend on capital letters or on how well a named-entity recogniser works.

For each entity I collected 100 passages from the November 2023 English Wikipedia dump, 6,000 in all. A passage qualifies if the entity appears in it exactly once, none of the other nine entities of its type appears, at least 50 tokens come before the entity, and 16 tokens come after it. Articles whose title contains the entity are skipped, so that the passage is about something else and merely mentions the entity, and each article contributes at most one passage per entity.

The passages are split per entity: 30 for testing, and of the remaining 70, 14 for validation and 56 for fitting. That gives 1,800 test, 840 validation and 3,360 fitting passages. Test passages are never used to fit a probe or to fine-tune anything.

3.2 Activations

For every passage I ran the target model and saved the layer-20 activation at six positions: the entity's last token, and 1, 2, 4, 8 and 16 tokens after it. I call these *distances*. Distance 0 is where the model has just read the entity; distance 16 is roughly a sentence later.

Before trusting any number, I checked that my pipeline reproduces known reconstruction quality. I had an earlier pilot run of the released models, written separately and run on different hardware, which gave a mean cosine of 0.907 between original and rebuilt activations. Repeating that run's recipe, the pipeline used here gives 0.908, and explains 75.0% of the variance, in line with the roughly 75% the authors report for this model. Getting there required fixing an off-by-one in which layer is read, described in Appendix B. On Wikipedia passages the mean cosine is 0.893, a little lower.

Type	Entities
Country	France, Japan, Egypt, Brazil, Canada, India, Germany, Italy, Mexico, Russia
City	Paris, Tokyo, Cairo, Chicago, London, Berlin, Moscow, Sydney, Toronto, Rome
Person	Shakespeare, Napoleon, Einstein, Mozart, Darwin, Lincoln, Churchill, Gandhi, Picasso, Beethoven
Animal	dog, cat, horse, lion, tiger, elephant, wolf, eagle, rabbit, dolphin
Food	bread, cheese, rice, pasta, chocolate, pizza, butter, honey, potato, sugar
Instrument	piano, guitar, violin, cello, flute, trumpet, saxophone, clarinet, harp, banjo

Table 3.1: The 60 entities.

3.3 Explanations

Generating explanations is the expensive step, so the main measurements use a fixed subset of the test set: nine test passages per entity, chosen by a seeded shuffle before any explanation was generated, at distances 0, 1, 4 and 16. That is 540 passages and 2,160 activations. For each activation the verbalizer wrote four explanations, sampled independently at temperature 1, for 8,640 explanations in total. Four samples per activation is what later makes it possible to ask whether the samples agree.

3.4 Name Detector

Every result depends on deciding, for an explanation and its passage, which names are true and which are wrong. The rules are:

- The explanation **names the true entity** if the entity, or one of a small fixed set of variants (*wolves* for wolf, *puppy* for dog), appears as a whole word.
- A **wrong name** is any other entity of the same type that appears in the explanation and does *not* appear in the passage. A same-type name that is in the passage is real content the explanation got right, so it is not counted against it.

"Any other entity of the same type" has to reach well beyond my ten. For countries I use every country in GeoNames, for cities every city above 300,000 people, for people the spans a named-entity recogniser tags as persons (spaCy's small English model [5]), and for animals, foods and instruments the corresponding branches of WordNet [6], keeping a word only where that sense is its most common one. The broad lists matter: about four in five of the wrong names found later are outside my list of 60 (Section 4.2).

The detector is imperfect, and I would rather say how than hide it. A hand check of 30 flagged explanations found four kinds of false alarm: category words (*bird*, *mammal*), meals (*dinner*), words for people in general (*humans*), and "persons" that were not proper nouns. The detector used throughout drops these. What remains is that the recogniser sometimes misses or invents a person, ambiguous words such as *bear* and *bass* are never

detected, and adjectives such as *French* never count as a country. These errors affect every condition I compare in the same way, since the same detector scores all of them, but they do mean the absolute rates carry more uncertainty than their intervals show.

3.5 Probe Baseline

A falling true-name rate could mean two things: the verbalizer is failing to report something the activation holds, or the activation no longer holds it. To tell these apart I need an independent estimate of what the activation holds.

I use a linear probe [1]. For each type t and distance δ, a ten-way logistic regression on the standardised activation $\tilde{h}$ predicts which of the ten entities of that type is in the passage:

$$p(e \mid h) = \mathrm{softmax}\Big(A_{t,\delta}\,\tilde{h} + b_{t,\delta}\Big)_e . \tag{3.1}$$

Its regularisation strength is chosen on the validation passages from nine values between 10^{-5} and 10^{3}, and the probe is then refit on fitting and validation passages together and scored on the test passages. Chance is 10%.

The probe is a floor on what the activation holds, not a ceiling: a better reader might find more. It also has an easier job than the verbalizer, since it chooses among ten known answers while the verbalizer writes free text. I return to this when comparing the two.

3.6 Statistics

Passages about the same entity are not independent, so every interval in this report is a 95% bootstrap interval that resamples the 60 entities, with 10,000 resamples [2]. When two conditions are compared on the same passages, I bootstrap the per-entity difference, which is much tighter than comparing two separate intervals. All sampling uses fixed seeds, and every result is labelled as coming from *natural* text (real passages) or from *planted* concepts (vectors I added by hand). Chapters 4 to 6 use natural text only.

4 Fading

This chapter measures the problem. It asks how often an explanation names the entity that is really in the passage, how often it names a wrong one, and how both change as the model reads on past the entity. All results are on natural text.

4.1 Fade Curve

Start with what the activation holds. Table 4.1 shows how well the entity can be read from the layer-20 activation at each distance, on all 1,800 test passages. At the entity's own token the probe is right 99.9% of the time. One token later it is right 59% of the time, and by 16 tokens it is down to 31%, still three times chance.

Part of this drop is simply true of the model. A transformer does not need to carry the entity forward at every position, because attention can look back at the token where it was read. An honest explanation of a later activation should therefore mention the entity less often.

The question is whether the explanation fades at the same rate. Figure 4.1 and Table 4.2 put the verbalizer next to the probe on the same 540 passages.

At distance 0 the two agree: the explanation names the entity in all but one of 2,160 cases. After that they part. At 4 tokens the probe reads the entity 43% of the time and the explanation names it 22% of the time. At 16 tokens it is 35% against 7.5%. The verbalizer is leaving on the table between half and four fifths of what a simple linear reader can still get.

This comparison is somewhat unfair to the verbalizer: the probe must answer every time, while the verbalizer can stay silent. Table 4.3 corrects for that by scoring the probe only on its most confident passages, as many as the verbalizer chose to name something in.

At 1 and 4 tokens, when the verbalizer names something it is about as accurate as the equally selective probe: 68% against 69%, and 48% against 51%. Its lower true-name rate there comes from speaking less often, which is defensible. At 16 tokens the two separate: the verbalizer's precision is 27%, the probe's is 53%. Far from the entity, when the verbalizer names something of the right type, it names the wrong thing three times out of four.

4.2 Wrong Names

The orange line in Figure 4.1 falling would be harmless on its own. What makes it a problem is the green line. At every distance a large share of explanations contain at least one wrong name of the entity's type: 45% at the entity's token, 54% one token later, and still 22% at 16 tokens (Table 4.2).

Tokens after entity	Concept-vector readout	Trained probe
0	49.1 [42.1, 56.2]	99.9 [99.7, 100.0]
1	30.2 [25.1, 35.5]	59.2 [55.3, 63.0]
2	27.2 [22.7, 31.8]	46.1 [41.7, 50.6]
4	25.8 [21.3, 30.6]	39.6 [35.4, 43.8]
8	22.8 [18.5, 27.4]	36.2 [31.7, 40.8]
16	22.4 [18.7, 26.5]	31.2 [26.9, 35.9]

Table 4.1: Reading the entity from the activation: top-1 accuracy (%) among the ten entities of the same type, with 95% intervals. Natural text, 1,800 test passages; chance is 10%. The concept-vector readout needs no training data and is discussed in Chapter 6.

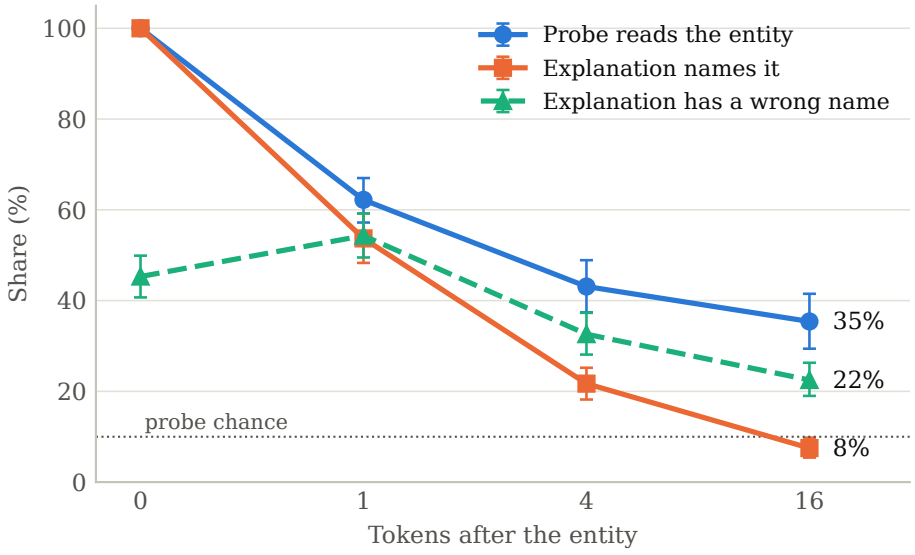


Figure 4.1: The specific fades faster from the explanation than from the activation. Natural text, 540 held-out passages, four explanations per activation; bars are 95% intervals over entities. The dotted line is the probe's chance level; it does not apply to the explanations, which are free text.

Two things stand out. First, wrong names are common even at distance 0, where the true name is always present. There the explanation typically names the right entity and then adds others around it. In one example about a recording of Beethoven's Spring sonata, the explanation correctly quotes "and Beethoven" but also invents "Mozart's complete piano sonatas" as the preceding item. Second, as the true name disappears the wrong names do not disappear with it. By 16 tokens, an explanation that names any entity of the type is far more likely to be naming only a wrong one (20% of all explanations) than the true one (7.5%).

The wrong names are not drawn from my list of 60. At 4 tokens, 77% of them are same-type entities outside the list, and at 16 tokens 82%. The most common are everyday members of the type: guitar, dog, piano and lion at 4 tokens; Toronto, piano, guitar and Italy at 16. They look like what one would guess knowing only the type. Here is a typical case, four tokens after *Moscow*:

Tokens after entity	Probe reads it	Names the true entity	At least one wrong name	Only wrong names	Wrong names per explanation
0	100.0 [100.0, 100.0]	100.0 [99.9, 100.0]	45.3 [40.7, 49.9]	0.0 [0.0, 0.1]	0.72
1	62.2 [57.2, 67.0]	53.7 [48.3, 59.2]	54.3 [49.5, 59.2]	25.1 [21.5, 28.9]	1.10
4	43.1 [37.4, 48.9]	21.7 [18.2, 25.2]	32.6 [28.1, 37.3]	23.5 [19.8, 27.2]	0.52
16	35.4 [29.4, 41.5]	7.5 [5.4, 9.8]	22.5 [19.0, 26.3]	20.0 [16.7, 23.6]	0.33

Table 4.2: What the explanations name, in % of explanations with 95% intervals. Natural text, 540 passages, 8,640 explanations. "Only wrong names" means the explanation names a wrong same-type entity and not the true one. The first column is the probe's accuracy on the same passages.

Tokens after entity	Names any same-type entity	Verbalizer precision	Probe, equally selective
0	100.0%	100.0 [99.9, 100.0]	100.0 [100.0, 100.0]
1	78.9%	68.1 [63.1, 72.9]	69.2 [64.1, 74.0]
4	45.2%	48.1 [41.7, 54.4]	50.8 [41.5, 60.9]
16	27.5%	27.2 [20.4, 34.2]	52.7 [42.4, 63.4]

Table 4.3: The verbalizer against an equally selective probe (%, 95% intervals; natural text). Precision is the share of naming explanations that name the true entity. The probe is scored on its most confident passages, as many as the verbalizer chose to name.

Passage (read to four tokens past *Moscow*)

> *. . . At nine, Forough was the youngest ever to be admitted to the Royal Conservatory of Music, Brussels as a pupil of Arthur Grumiaux. Later studies were with David Oistrakh at the* ***Moscow*** *Conservatory. . . .*

Explanation

> *Wikipedia biographical article format with English language entries on Ukrainian tenor Vladimir Ashkenazy, detailing career and musical achievements. The sentence structure "Ashkenazy studied music in* ***Paris*** *beginning in 1952, and won international competitions . . . "*

The gist is intact: a biography of a classical musician with Soviet connections, studying at a conservatory in a capital city. The city is wrong, and so is the musician.

4.3 Name Position

Each explanation describes the context and then predicts what comes next. A wrong name in the prediction ("expecting *Paris* or *London*") is a guess labelled as a guess. A wrong name in the description of the context is a false statement about what the model read. Table 4.4 separates the two, cutting each explanation at the phrase that introduces its predicted continuation.

Most wrong names are in the description, not the prediction. One token after the

Tokens after entity	True name, context	True name, continuation	Wrong name, context	Wrong name, continuation
0	100.0 [99.9, 100.0]	11.1 [8.1, 14.6]	35.8 [32.2, 39.4]	17.9 [14.5, 21.5]
1	53.1 [47.5, 58.6]	7.8 [5.9, 9.8]	51.7 [47.0, 56.4]	15.3 [12.5, 18.2]
4	20.5 [17.0, 24.0]	3.2 [2.0, 4.5]	30.2 [26.0, 34.6]	7.7 [6.0, 9.5]
16	6.7 [4.7, 8.8]	1.4 [0.7, 2.1]	20.8 [17.4, 24.4]	4.8 [3.5, 6.2]

Table 4.4: Where names sit in the explanation: % of explanations with a true or wrong name in the part that describes the context, and in the part that predicts the continuation. Natural text; 95% intervals.

entity, 52% of explanations have a wrong name in the context part and 15% have one in the continuation part; in raw counts, 1,993 wrong names against 495. The ratio is between two and five to one at every distance. The problem is therefore not that the verbalizer hedges with a list of candidates for the next word. It states, as a description of the text, things that are not in the text.

In short, the specific fades faster from the explanation than from the activation, and what takes its place is a plausible wrong name stated as fact.

5 Incentives

The verbalizer was trained to maximise one number, the reconstruction cosine. If wrong names cost it nothing, training never had a reason to remove them. This chapter tests that directly.

5.1 Edit Test

I took explanations at distances 0 and 1 that name the true entity and also contain at least one wrong name in the part describing the context. There are 1,352 of them; I sampled 1,000. For each I made two edited copies and sent all three through the reconstructor.

- **Wrong names removed.** Every wrong same-type name is replaced by a neutral phrase such as "another city". The true name stays.
- **True name removed.** The true name is replaced the same way. The wrong names stay. This is the control: it shows what an edit of the same size costs when it removes real information.

Writing z^{-W} for the copy with wrong names removed and z^{-e} for the copy with the true name removed, the quantities of interest are what each kind of name is worth to the score of Equation 2.1:

$$\Delta_{\text{wrong}} = s(h, z) - s(h, z^{-W}), \qquad \Delta_{\text{true}} = s(h, z) - s(h, z^{-e}). \tag{5.1}$$

If $\Delta_{\text{wrong}} \leq 0$, the objective in Equation 2.2 would already discourage wrong names. If it is near zero or positive, it does not.

The edits worked as intended: afterwards no wrong name is detected in 99.8% of the first copies, and the true name is gone from 94.3% of the second.

5.2 Results

Δ_{wrong} is 0.003 at the entity's token and 0.006 one token later (Table 5.1, Figure 5.1); per name removed, about 0.002. Δ_{true} is 0.211 at the entity's token, seventy times as much, and 0.016 one token later.

5.3 Interpretation

Three things follow.

Wrong names are nearly free, and slightly rewarded. The original beats the cleaned copy 68% of the time at distance 0 and 80% at distance 1. A wrong but plausible name of the right type reconstructs a little better than a vague phrase, presumably

Tokens after entity	n	Original	Wrong names removed	True name removed	Original beats "wrong removed"	Original beats "true removed"
0	567	0.897 [0.892, 0.902]	0.894 [0.889, 0.899]	0.686 [0.676, 0.695]	67.5 [63.0, 72.1]	100.0 [100.0, 100.0]
1	433	0.895 [0.886, 0.903]	0.889 [0.880, 0.897]	0.879 [0.869, 0.887]	79.9 [75.5, 84.2]	88.2 [84.1, 92.0]
0 and 1	1,000	0.896 [0.890, 0.901]	0.892 [0.886, 0.897]	0.769 [0.759, 0.779]	72.9 [69.6, 76.1]	94.9 [93.1, 96.6]

Table 5.1: Reconstruction cosine before and after editing names out, and how often the original scores higher than each edit (%). Natural text; 95% intervals over passages.

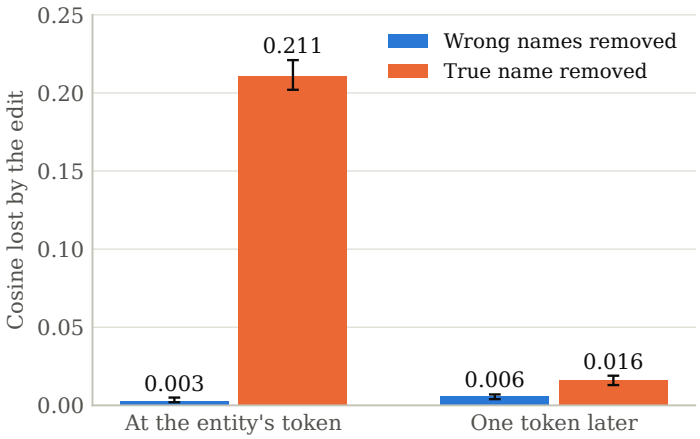


Figure 5.1: What each kind of name is worth to the reconstruction score: cosine lost when it is edited out. Natural text; 95% intervals.

because it still tells the reconstructor the type and the topic. So the objective gives the verbalizer a small push towards naming *something*.

The true name stops mattering quickly. It is worth 0.21 while the model is reading it and 0.016 a single token later. From then on, getting the specific right earns almost nothing, so a verbalizer trained only on reconstruction has little reason to keep track of it.

This sets the scale for a fix. A penalty of λ per suspect name changes the reward to $s(h, z) - \lambda\,|U(z)|$, where $U(z)$ is the set of names to discourage. For it to bite without also driving out true names, λ must sit between what a wrong name earns and what a true one earns:

$$0.002 \;\approx\; \Delta_{\text{wrong}} \text{ per name } \;<\; \lambda \;<\; \Delta_{\text{true}} \;\approx\; 0.016 \text{ (one token after the entity).} \quad (5.2)$$

The fine-tuning experiments use $\lambda = 0.01$.

The verbalizer is doing what an autoencoder does: it keeps what helps reconstruction. The specific helps very little, so nothing protects it.

6 Agreement

If the reconstruction score cannot tell a true name from a wrong one, something else must. This chapter shows that the verbalizer's own samples can: a name it writes every time is usually true, and a name it writes once is usually not.

6.1 Consensus

Each activation has $K = 4$ independently sampled explanations $z_1, \ldots, z_K$. For every name n that appears in any of them, I count how many contain it:

$$c(n) = \sum_{k=1}^{K} \mathbf{1}\Big[\, n \in N(z_k) \,\Big]. \tag{6.1}$$

Figure 6.1 shows how often a name is true at each count, pooling distances 1, 4 and 16. Distance 0 is left out of the pooled figure because the true name is always present there, which would flatter the result.

A name given by one sample is true 6% of the time. A name given by all four is true 64% of the time. As a score for ranking names, the agreement count has an AUC of 0.80, and it holds up at every distance (Table 6.1).

6.2 Filter

This suggests a filter that needs no training: keep a name only if $c(n) \geq m$ for some threshold m. Table 6.2 shows the trade-off in the terms of retrieval, treating the names as retrieved items.

Requiring two of four removes 74% of the wrong names and keeps 77% of the true ones, which doubles precision from 17.5% to 38%. Requiring all four reaches 64% precision but keeps only half the true names. Two of four is the setting I carry forward: it removes most of the damage for a modest loss.

The cost is compute. Four explanations instead of one is four times the generation. The filter also only removes names; it cannot recover a true name the verbalizer never wrote.

6.3 Verifiers

Agreement uses only the verbalizer's outputs. An alternative is to check each name against the activation itself: does the activation point towards the named entity more than towards its rivals? Table 6.3 compares three such checks.

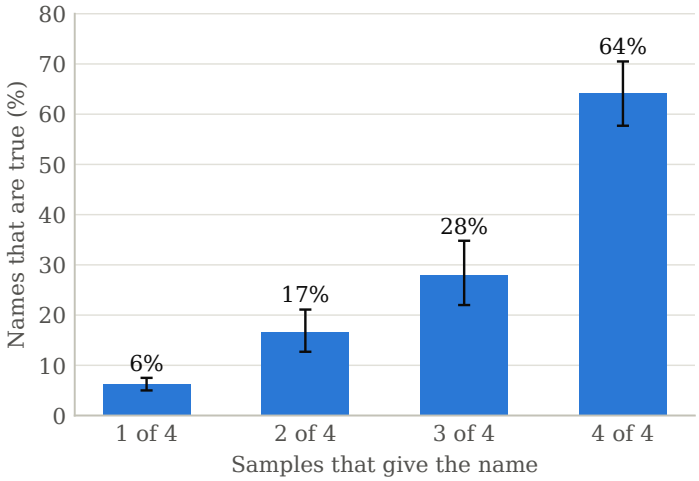


Figure 6.1: Names that more samples agree on are more often true. Natural text, distances 1, 4 and 16 pooled, 3,489 names; 95% intervals.

Tokens after entity	1 of 4	2 of 4	3 of 4	4 of 4
0	0.0 [0.0, 0.0] (717)	0.0 [0.0, 0.0] (148)	1.3 [0.0, 3.8] (78)	87.5 [84.8, 90.3] (616)
1	5.2 [3.8, 6.8] (1,113)	14.2 [10.1, 18.4] (240)	20.3 [14.1, 27.3] (182)	66.4 [59.7, 73.0] (348)
4	7.8 [5.9, 10.0] (667)	20.0 [11.5, 30.0] (115)	39.3 [28.9, 50.7] (84)	62.4 [53.0, 71.7] (109)
16	6.1 [4.1, 8.5] (478)	19.3 [11.4, 28.4] (83)	39.0 [21.4, 57.9] (41)	44.8 [26.9, 62.1] (29)
1, 4 and 16	6.2 [5.0, 7.5] (2,258)	16.7 [12.7, 21.1] (438)	28.0 [22.0, 34.8] (307)	64.2 [57.7, 70.5] (486)

Table 6.1: Precision of a name (%) by how many of the four samples give it, with the number of names in brackets. Natural text; 95% intervals.

- **Concept vectors** are entity directions built without any labelled passages, from the target model's activations on the prompt "Tell me about X". They are weak: AUC 0.65 to 0.79.
- **Probe directions** come from the trained probe. They are the best check near the entity (0.90 at one token), but they need dozens of labelled passages for every entity one wants to verify.
- **Agreement** needs neither labels nor access to the target model. It matches the probe directions from four tokens on (0.77 against 0.76, and 0.72 against 0.70).

For a method whose selling point is that it is unsupervised, agreement is the natural check: it is nearly as good as the supervised one and works for any name, not just the entities someone trained a probe for.

Keep a name if given by	Wrong names removed	True names kept	Precision	Names left
None	0	100	17.5 [15.1, 20.1]	3,489
At least 2 of 4	73.6 [69.9, 77.3]	77.2 [73.5, 80.6]	38.3 [33.5, 43.4]	1,231
At least 3 of 4	86.3 [83.9, 88.6]	65.2 [61.6, 68.9]	50.2 [44.6, 56.2]	793
All 4	94.0 [92.7, 95.2]	51.1 [46.3, 55.9]	64.2 [57.7, 70.5]	486

Table 6.2: The agreement filter as a precision–recall trade-off (%). Natural text, distances 1, 4 and 16 pooled; 95% intervals. "True names kept" is recall relative to no filter.

Tokens after entity	Concept vectors	Probe directions	Agreement count
0	0.79 [0.74, 0.84]	1.00 [1.00, 1.00]	0.96 [0.95, 0.97]
1	0.69 [0.63, 0.74]	0.90 [0.86, 0.92]	0.83 [0.80, 0.86]
4	0.66 [0.57, 0.73]	0.76 [0.69, 0.84]	0.77 [0.74, 0.80]
16	0.65 [0.54, 0.76]	0.70 [0.61, 0.78]	0.72 [0.66, 0.78]

Table 6.3: Three ways to tell true names from wrong ones (AUC, 95% intervals; natural text). The first two score a name by the activation's similarity to a direction for that entity; they cover only names among the 60 listed entities, so the columns are not computed on identical sets of names.

Bibliography

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

[2] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[3] Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarng, Alex Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lindsey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce unsupervised explanations of LLM activations. *Transformer Circuits Thread*, 2026. URL https://transformer-circuits.pub/2026/nla/index.html.

[4] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.

[5] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python. Zenodo, 2020.

[6] George A. Miller. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41, 1995.

[7] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[8] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Yang Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.